

技术一把手用 Matt Pocock Skills 提升团队效率

原创 物理知识点 物理知识点 2026年8月19日 09:12 湖南



“你管着十几二十号人，每天不是救火就是在救火的路上。需求理解不一致、Code review 标准不一、Bug 反复出现、合并冲突天天有——你缺的不是技术方案，你缺的是一套团队都认可、可重复执行的工作流程。”

01 先说说——技术管理者每天都在跟什么较劲

你是一个技术负责人，你管着十几二十号人。

每天早上打开消息，钉钉群里几十条未读。有人问接口怎么设计，有人在吵代码要不要拆模块，还有人喊合并冲突了。你开站会、定规范、做 code review。你觉得干得不错，但总有那么几个问题反复出现：

-  **新同学写的代码和现有风格完全不搭**
review 了三轮，改了三轮，最后还是你亲自上手改的
-  **两个组对同一个术语的理解不一样**
接口联调的时候才发现鸡同鸭讲，返工两天
-  **线上出 Bug，一窝蜂去看代码**
猜了一下午，最后发现是极简单的边界条件
-  **合并冲突一出现，新人不敢动老人不想动**
项目进度卡在那，所有人都在等

这些问题单拎出来都不大，但它们天天出现，消耗的是团队最宝贵的资源——注意力。说白了，你缺的不是技术方案，你缺的是一套团队都认可、可重复执行的工作流程。

02 Matt Pocock Skills 是什么？不是什么？

Matt Pocock，TypeScript 社区有名的教育家，Total TypeScript 的创始人。他设计了一套技能系统，把软件开发从想法到交付的整个过程拆成了一个个独立的“技能”。每个技能在 Claude Code 里以斜杠命令的形式调用，比如 `/code-review`、`/tdd`。

你可能看过从开发者视角写的文章，告诉你每个技能怎么用。但今天换个角度——**从技术管理者的视角来看**。这些技能不是给一个程序员用的，是给你整个团队用的。

**核心问题**

怎么让团队里每个人写代码的方式基本一致，减少不必要的沟通成本，把精力花在真正重要的事情上？

技能	管理者拿来干什么	解决团队什么问题
/grill-with-docs	需求澄清标准化	需求理解不一致，反复返工
/domain-modeling	统一团队语言	术语混乱，联调沟通成本高
/codebase-design	模块设计原则	代码结构混乱，新人上手慢
/tdd + /implement	质量内建	测试覆盖不全，上线频繁出 Bug
/code-review	自动化审查标准	Code review 标准不一，漏审
/diagnosing-bugs	系统化调试流程	Bug 靠直觉猜，修了又复发
/resolving-merge-conflicts	减少合并阵痛	合并冲突耗费大量时间
/research + /prototype	调研验证委托	方向不确定就投入大量开发资源

03 需求澄清 + 统一语言——从源头消灭"我以为"

你肯定遇到过这种事：产品经理说了一个需求，开发说"懂了"，做出来发现完全不是一回事。返工、扯皮、延期。问题出在"懂了"这两个字上。每个人脑补的东西不一样，但没人意识到这点。

/grill-with-docs——不会累的追问机器人

这玩意儿不接受"大概明白了"这种回答，而是一个接一个地问问题，直到把需求里的每个模糊点都搞清楚。每问一个，它就把确定下来的术语写进项目文档里。

```
# 跑完 /grill-with-docs 后产出的术语表
# 自动写入 CONTEXT.md

**Task**: 一个待办事项，包含标题、描述、状态
**PomodoroSession**: 一次番茄钟工作会话
**TimerState**: 计时器状态（空闲/工作中/休息中/暂停）
```

Matt Pocock 原话是这么说的：*"It refuses to hypothesise before you have a tight feedback loop."* 大白话就是——别猜，确定下来再干活。

**管理者实操**

把 /grill-with-docs 当作新人入职的"标准作业"。不管谁接手需求，第一件事不是写代码，是跑一遍这个技能，把产出的 CONTEXT.md 扔到群里。所有人对需求的理解从一起跑线开始。

/domain-modeling——同一个词只能有一个意思

你团队里，"订单"这个词可能同时指：用户下单的动作、数据库里的一条记录、前端展示的卡片、支付回调里的对象。这些东西不完全一样，但大家都在用同一个词说不同的事。

开会没人觉得有问题，写代码的时候全暴露了。/domain-modeling 就是干这活的——揪出一词多义，帮团队建立统一的领域语言。每个术语有且只有一个含义，写进 CONTEXT.md，所有人都得遵守。而且它会记录架构决策（ADR），比如“番茄钟被中断时暂停而非终止”这种影响整个架构的决策。

💡 管理者实操

新项目启动时，专门开一个小时的“术语对齐会”跑一遍这个技能。产出的术语表贴在项目 Wiki 首页。新同学加入项目，先看术语表，再读代码。跨团队协作尤其管用。

04 模块设计 + 质量内建——让新人也能写出好代码

老员工写的模块，接口清晰，内部复杂但调用者不需要知道。新同学写的模块，一个文件 500 行，函数之间互相调用，拆都拆不开。你没法天天盯着每个人怎么写代码，你需要的是标准和流程。

/codebase-design——一套团队能讨论的设计语言

它提供了一套核心概念，让团队能用统一的词汇讨论模块设计：



Deep Module (深模块)

接口小而简单，内部实现复杂——好模块的标志



Seam (接缝)

不改代码就能改变行为的地方——可测试性的关键



Leverage (杠杆)

调用者每学一个接口能获得多少功能——杠杆越高越好



Deletion Test

想象删除这个模块，如果复杂度没消失，说明它在干活

💡 管理者实操

Code review 的时候用这套语言沟通。不说“这个模块写得不好”，说“这个模块的杠杆太低了，调用者需要了解三个内部函数才能用”。新人也能参与设计讨论。

/tdd——先写测试，再写代码，一次只干一件事

你定了 KPI 要求测试覆盖率达到 80%，结果大家写了套“为了覆盖率而写”的测试——测的都是 getter/setter，真正的业务逻辑一个没测。传统“先写代码再补测试”的流程，测试是附加的、可选的、可以糊弄的。

/tdd 的做法完全反过来——先写测试，再写代码，一次只写一个测试。这叫“红-绿-重构”循环：写一个测试让它红（失败），加刚好够的代码让它绿（通过），然后重构。

```
// 第一个测试：证明单条路径端到端可用
it('可以开始一个番茄钟', () => {
  const timer = createTimer({ duration: 25 * 60 })
  timer.start()
  expect(timer.state.status).toBe('working')
  expect(timer.state.elapsed).toBe(0)
})
```

```
// 写第二个测试之前，确保第一个通过了
it('可以暂停一个工作中的番茄钟', () => {
  const timer = createTimer({ duration: 25 * 60 })
  timer.start()
  timer.pause()
  expect(timer.state.status).toBe('paused')
})
```

注意期望值——‘working’和‘paused’是手动写进去的，不是从代码里算出来的。Matt Pocock 强调了一个原则：**期望值必须来自独立的事实来源**。不能用代码自己的计算结果来验证自己，那是同义反复测试，通过了也不告诉你任何东西。

💡 管理者实操

把 `/tdd` 作为 Bug 修复的标准流程。任何 Bug 修复必须先用 `/diagnosing-bugs` 建一个可复现的测试，再用 `/tdd` 修复。这样每个 Bug 修复都自带回归测试，同一个 Bug 不会出现两次。

05 审查 + 调试 + 冲突——把“救火”变成“防火”

`/code-review`——两个维度独立审查，不合并评分

Code review 可能是你花时间最多、但效果最不确定的管理动作。你亲自 review 累；你让 peer review，大家互相放水。你定了一堆 review checklist，没人看。

`/code-review` 沿着两个独立的维度审查代码：**Standards（编码标准）**和**Spec（需求规格）**。两个维度由两个独立的子 agent 分别评估，结果分开呈现。因为一个改动可能标准很好但规格不对，或者规格对了但代码很烂。混在一起报告会掩盖问题。

```
## Standards
- ✅ 命名清晰，符合项目规范
- ❌ timer.ts:45 handleTick 方法过长（48 行），建议拆分
- ❌ session.ts: 缺少对异常状态的处理

## Spec
- ✅ 实现了番茄钟 25 分钟工作 / 5 分钟休息
- ✅ 支持暂停和恢复
- ❌ 缺少“每 4 个番茄钟后强制长休息”（spec 要求）
- ✅ 没有引入未要求的额外功能
```

💡 管理者实操

把 `/code-review` 接入 CI 流水线。每次 PR 提交后自动跑一遍，审查结果贴在 PR 评论里。开发者先看 AI 的审查结果修一轮，再找真人 review。基础问题被筛掉了，reviewer 只需要关注真正的设计问题。你作为管理者也不用亲自做基础审查了。

`/diagnosing-bugs`——第一步不是读代码，是建测试

线上出 Bug 了，团队的反应通常是：一群人冲进代码里翻，找了一下午，各说各的猜测。最后有人说“试试这样改”，改完好像好了，但谁也不知道为什么。过两周同样的 Bug 又出现了。

这个技能最反直觉的原则是：**第一步不是读代码，而是构建一个可复现的测试**。在能稳定复现 Bug 之前，不允许提任何假设。Matt Pocock 原话：*"It refuses to hypothesise before you have a tight feedback loop — one runnable command that already goes red on this bug."*

```
// 先构建 tight feedback loop
it('在计时结束瞬间点击暂停，不会进入异常状态', () => {
  const timer = createTimer({ duration: 0.01 })
  timer.start()
  timer.pause()
  const state = timer.state
  expect(['completed', 'idle']).toContain(state.status)
})
```

新手容易踩的坑：调试的时候加了一堆 `console.log`，修完之后忘了删。调试日志标记为 `[DEBUG-xxx]` 格式，修完后 `grep [DEBUG` 搜一遍清理掉。

💡 管理者实操

定一条团队规则：“**报 Bug 必须附带复现步骤**”。不提供复现步骤的 Bug 报告打回重写。这个规则会倒逼团队建立更好的 Bug 报告习惯。

/resolving-merge-conflicts——按意图解决，不按文本解决

合并冲突这玩意儿，是团队协作的“显影剂”。冲突越多，说明协作摩擦越大。很多团队的处理方式是：谁先碰到谁解决，解决的方式是把两边代码都看一遍，然后手动拼起来。

这不对。合并冲突不应该按“文本”解决，应该按“意图”解决。这个技能的做法：先查两边提交的意图——你改了啥、为什么改，对方改了啥、为什么改。理解意图之后，再决定冲突区域怎么处理。而且它永远不会 `--abort`，一定会把合并进行到底。

```
// 冲突区域：
// 你的改动：在 pause() 里加了 completed 状态检查
// 同事的改动：把整个逻辑搬到了 StateMachine 类里
// 解决方案：把 completed 状态检查加到 StateMachine 的 pause 方法里
// 两者的意图都被保留了
```

💡 管理者实操

新人遇到合并冲突，先跑这个技能生成解决方案，再拿着方案去找相关同事确认。既保证了准确性，又让新人学到了“按意图解决”的思路。另外，统计哪些文件经常冲突——那些文件就是需要重构的信号。

06 调研验证 + 落地计划——别开会，用数据说话

团队经常遇到这种场景：要不要用这个新框架？这个方案性能怎么样？然后大家开会讨论了半天，没有结论，因为没有数据。

/research + /prototype——两个小时后就有答案

`/research` 跑在后台，自动去查一手资料，返回一份带引用的调研报告。它只读官方文档、源代码、规范说明书，不读二手总结。你可以在它跑的时候干别的活。

`/prototype` 更进一步——创建一个可丢弃的小程序，只回答一个设计问题。代码放在侧分支上，不进入主分支，没有测试，没有错误处理。代码里加一句注释：

```
// PROTOTYPE: 回答“状态机手感对吗？”——用完即弃
```

 管理者实操

当团队在讨论“该不该用 X”的时候，别开会。直接派一个人跑 `/research` 出调研报告，再跑 `/prototype` 验证核心场景。两个小时后就有数据了。用数据说话，而不是用嗓门说话。

五阶段落地计划——从最容易的开始

道理都懂，怎么让团队用起来？不能一次性推全部技能，选两三个最痛的点，先推起来。

1 第一阶段（第 1 周）
`/code-review` 接入 PR 流程。最没有争议，因为它是“加一道保障”而不是“改变工作方式”。

2 第二阶段（第 2-3 周）
`/grill-with-docs` 作为需求澄清标准步骤。选一个正在启动的项目做试点。

3 第三阶段（第 4 周起）
`/diagnosing-bugs` 处理线上 Bug。定规则：修 Bug 必须有复现测试。

4 第四阶段
`/tdd` 在新模块中试点。选一个不太紧迫的新模块，让感兴趣的同事先试。

5 第五阶段
剩下的技能按需引入。

几个实操建议，你琢磨琢磨：

管理者带头用——你自己写 code review 的时候先跑一遍 `/code-review`，你处理需求的时候先跑 `/grill-with-docs`。团队看到你用，才愿意跟。

不搞强制——别发通知说“以后所有人必须用 XXX”。找个志愿者先试，出效果了再推广。

记录成果——用之前和用之后做个对比。比如“接入 `/code-review` 后，PR 平均 review 轮数从 3.2 降到了 1.5 轮”。有数据才好说服人。

定期回顾——每个月花半小时聊聊技能用得好不好，哪些地方需要调整。技能是给人用的，不是供起来的。

07 诚实地说——哪些技能还不够火候

这套技能系统虽然好，但有些地方得说实话，不是所有技能都适合所有团队：

 **`/tdd` 需要先有测试文化**
如果团队连单元测试都没写过，直接上 TDD 会适得其反。建议先让团队习惯写测试，再引入 TDD 流程。

 **`/domain-modeling` 的收益需要时间显现**
术语统一的效果在前两周不明显，等到跨团队联调的时候才体现出来。管理者要有耐心，别做了一周就放弃。

**/resolving-merge-conflicts 依赖团队契约**

如果团队习惯"改代码不清分支"、"commit message 随便写"，这个技能的效果会大打折扣。它按意图解决冲突，前提是提交信息有意图可查。

💡 我的看法

技术管理者的工作，说白了就两件事：减少团队的无效劳动，放大团队的有效产出。Matt Pocock Skills 不是给你一个更好的编辑器或者更好的框架，它给你的是一套**团队层面的工作方法**。你不需要记住每个技能怎么用，只需要记住一件事：**团队遇到什么类型的重复问题，就引入对应的技能来解决它**。从"每次出问题都去救火"，变成"用一套流程让火根本烧不起来"。有一说一，这才是技术管理者的终极目标。

**小测验****Q1: /diagnosing-bugs 的第一步是什么？**

👉 点击每个选项查看对错

A. 阅读代码，构建理论假设

B. 构建可复现的测试 (tight feedback loop)

C. 直接修复看起来有问题的地方

Q2: /code-review 沿着哪两个维度审查？

👉 点击每个选项查看对错

A. 性能和安全性

B. 前端和后端

C. Standards (标准) 和 Spec (规格)

**笔记完成**

理解了从技术管理者视角使用 Matt Pocock Skills 的方法、
每个技能对应的团队协作场景、五阶段落地计划、
以及从"救火"到"防火"的管理思维转变。

📄 项目地址: github.com/mattpocock/skills · Total TypeScript: totaltypescript.com

📖 推荐阅读: 上一课《Matt Pocock Skills 全流程工作流指南》——开发者视角

其他分类 · 目录

上一篇

DeepSeek Harness 72.3K Stars! "一切皆插件"的 Agent 框架, 把 Claude Code 和...

下一篇

Semantica 9.2K Stars! 开源 Palantir, 给 AI 装上决策大脑和审计日志